

COMP1850 Programming

School of Computer Science, University of Leeds

Worksheet - Week 1.4

This worksheet contains a combination of formative activities and activities that contribute towards your portfolio.

- Portfolio activities are indicated in the title of the task.
- Activities marked by (*) are advanced activities and may take more time to complete.

Expectations:

1. Timeliness: You should complete all the tasks in the order provided and submit your portfolio task to Gradescope before the deadline.
2. Presentation: You should present all your work clearly and concisely following any additional guidance provided by the module staff in the module handbook.
3. Integrity: You are responsible that the evidence you submit as part of your portfolio evidence is **entirely your own work**. You can find out more about Academic integrity on the Skills@library website. All work you submit for assessment is subject to the academic integrity policy.

Feedback: Feedback on formative activities will be provided via Lab classes and tutorials. Feedback on evidence submitted as part of the portfolio will be available on Gradescope.
Support opportunities: Support with the activity sheet is available in the Lab classes and tutorials. Individual support is available via office hours.
Expected complete date: Friday 30th October

Within this assessment you **may** use Generative AI to:

- help you start to understand more complex ideas by providing accessible summaries
- test your knowledge against a piece of content
- help identify and correct spelling mistakes and grammatical errors in your work
- provide feedback and advice on your overall coding style

You must **not** use Gen AI to:

- produce written content (code, comments or any other content) which you then submit as your work, regardless of whether you make changes to it.
- rewrite or make changes to any of your work.

Learning Outcomes:
On completion of this activity sheet you will:

- Write comments for a program or a module specifying what it does

- Develop functions with parameter passing

- Develop recursive functions

Task 1

1.1 Given the following function definition and docstring, answer the following questions.

```
def calculate_travel_distance(tank_capacity, km_per_litre):
    '''
    This function calculates the distance a vehicle can travel based on tank
         capacity (ie how many litres of fuel in tank) and the number of
         kilometres a vehicle can travel per litre, and returns the
         distance, rounded to two decimal points.
    '''
    f_capacity = float(tank_capacity)
    f_kml = float(km_per_litre)
    distance = round(f_capacity * f_kml, 2)
    return distance
```

a. Will the program still run if an integer is passed to either tank_capacity or km_per_litre, eg print(calculate_travel_distance(10, 3)) or print(calculate_travel_distance(20.5, 2))? Explain.
b. Will the program still run with the following function calls? Please explain.
   i. print(calculate_travel_distance(tank_capacity=25, 2.0))
   ii. print(calculate_travel_distance(30.2, km_per_litre=3.3))

1.2 Given the following function definition, answer the following questions.

```
def func_1(lst_value):
    '''
         write your docstring to explain the function
    '''
    sum = 0
    num_of_value = len(lst_value)
    for value in lst_value:
              sum = sum + value
     return sum/num_of_value
```

a) Examine the code and write a dosctring for that function.
b) What would be a meaningful name for func_1?

1.3 You are given the following incomplete function definition of a recursive function.

```
def countdown(number):
    '''
    This function takes a positive integer number
    and counts down from the number to zero, printing each value
    '''
    print(number)
    countdown(number-1)
```

Complete and test the recursive function by including a base case at an appropriate point.

Task 2 – Portfolio task

Create a function named currency_converter that converts between four different currencies. The function should accept three arguments:

1. *amount*: A number representing the monetary value to be converted. It can be an integer or a floating-point number, and it must always be non-negative.
2. *from_currency*: A string representing the currency code of the original currency. The supported currency codes are:
   - 'GBP' (British Pound)
   - 'CNY' (Chinese Yuan)
   - 'PHP' (Philippine Peso)
   - 'INR' (Indian Rupee)
3. *to_currency*: A string representing the currency code to which the amount should be converted. The possible values for this argument are the same as for from_currency.

Expected Behavior:

- The function should return the converted amount as a floating-point number, rounded to two decimal point precision using Python's round() function.
- If the input amount is negative, the function should return 0.0.
- You do not need to handle cases where non-numeric values are passed in.
- You do not need to handle cases for undefined currency codes.

The conversion rates are stored in the following dictionary structure:

```
conversion_rate = {
'GBP': {'CNY': 9.24, 'PHP': 75.00, 'INR': 109.06},
'CNY': {'GBP': 0.11, 'PHP': 8.12, 'INR': 11.81},
'PHP': {'CNY': 0.12, 'GBP': 0.01, 'INR': 1.45},
'INR': {'PHP': 0.69, 'CNY': 0.09, 'GBP': 0.01}
}
```

The numbers in the inner dictionary are the conversion rates. For example, the conversion rate from GBP to CNY is 9.24. Therefore, 101 GBP will be converted to 933.24 CNY.

A template file currency_converter.py is provided and you are to write your function in this file. Please do not change the file name. **When submitting your file, make sure there are no additional function calls to test the function, or code to get user input. Only a file with the function definition is required.** The autograder on Gradescope will run tests and provide feedback on passed and failed tests.

You must pass all tests on Gradescope to complete the task.

Task 3 (*)

3.1 Integer multiplication can be broken down into repeated addition, as a recursion in the form

```
m x n = m + (m x (n-1)).
```

Therefore, we can compute the multiplication by repeatedly calling the function, each time with `n` decreasing by 1. The function stops calling itself when `n` reaches 1.

Write a recursive function for integer multiplication that accepts two parameters `x` and `y` and returns the result.

3.2 Write a function `unique_combinations(nums, target)` that takes a list of four integers (i.e. `nums`) and a `target` number. The function should return the first unique combination that produced `target` using all the integers in `nums` with a combination of the four basic arithmetic operators (+, -, *, /).

The combination needs to use the integers based on their order in `nums` and each can only be used once. Each of the basic arithmetic operators can only be used once in each combination.

Each unique combination is a mathematical expression returned as a string. For example, for `nums=[2, 3, 6, 7]` and `target=11`, the function might return `'2/3*6+7'`. The function returns an empty string if no solution can be found.

Sample Exam Question

1. State two differences between a built-in function and a user-defined function in Python.
2. State the purpose of the return statement in a function in Python.
3. State what happens if you do not specify a return statement in a Python function.